

Taller: Estructuras de Datos y Complejidad

Parte 1 — Complejidad algorítmica (Big O)

1.1 Clasifica la complejidad

Buscar una palabra en un diccionario físico (mitad de páginas): $O(\log n)$. Al partir las páginas por la mitad en cada intento, descartas de un solo golpe la mitad del libro. El volumen de datos se reduce tan rápido que encuentras la palabra en muy pocos pasos.

Revisar cada carné uno por uno: $O(n)$. Si la caja dobla su tamaño, el esfuerzo se dobla. En el peor de los casos, el carné que buscas está al final de todo, así que el tiempo crece al mismo ritmo que la cantidad de tarjetas.

Sacar la primera carta de un mazo barajado: $O(1)$. Es una acción instantánea. No importa si el mazo es de 40 cartas o de 10,000; meter la mano y agarrar la carta que está encima toma exactamente el mismo esfuerzo.

Comparar cada estudiante con todos los demás: $O(n^2)$. Esto requiere un cruce total. Para emparejar a todos, el primer alumno debe compararse con los demás, luego el segundo con los demás, y así sucesivamente. Básicamente, es meter un ciclo dentro de otro, lo que dispara el número de operaciones de forma cuadrática.

Consultar la hora en tu reloj: $O(1)$. Mirar la muñeca para ver la hora toma una fracción de segundo constante. El paso del tiempo o el tamaño del reloj no cambian lo que tardas en hacer esa lectura.

1.2 De la vida real al análisis

Descripción de la tarea: Calificar exámenes escritos a mano uno por uno. Si me toma 3 minutos revisar el examen de un alumno, calificar a un grupo entero dependerá directamente de cuántos estudiantes asistieron.

¿Qué pasa con $10n$? Si en lugar de calificar a 15 alumnos (n) tengo que calificar a 150 ($10n$), el tiempo total se multiplicará exactamente por 10 (pasando de 45 minutos a 450 minutos), ya que cada examen requiere el mismo esfuerzo individual.

1.3 ¿Cuál escala mejor?

Recomendación: La Forma B ($O(n \log n)$).

Justificación: Aunque para 5 datos la Forma A sea más rápida por milisegundos, diseñar software pensando solo en el presente es un error peligroso. Si el día de mañana los datos crecen a miles o millones, el algoritmo $O(n^2)$ colapsará el sistema porque sus operaciones crecerán

exponencialmente. En cambio, la Forma B se mantiene estable y eficiente sin importar cuánto crezca el volumen de información.

1.4 Verdadero o falso

a. Un algoritmo $O(1)$ siempre es más rápido en segundos reales: Falso.

Big O no mide segundos de reloj, sino cómo reacciona el algoritmo cuando los datos crecen. Un proceso $O(1)$ podría tardar 5 segundos fijos por ser muy pesado internamente, mientras que un proceso $O(n)$ con un solo dato ($n=1$) podría tardar apenas 0.01 segundos. El $O(1)$ solo ganará cuando el volumen de datos sea muy grande.

b. Big O describe cómo crece el tiempo de ejecución, no el tiempo exacto: Verdadero.

Su objetivo es medir la eficiencia y escalabilidad teórica a largo plazo, abstrayéndose de qué tan rápida sea la computadora o el procesador donde se ejecuta.

c. Buscar por índice en un arreglo es $O(n)$ porque hay que recorrerlo: Falso.

Es $O(1)$. Las computadoras no caminan el arreglo casilla por casilla para llegar a una posición. Usan una fórmula matemática interna que calcula la ubicación exacta en la memoria y saltan directamente a ella de forma inmediata.

Parte 2 — Arreglos 1D

2.1 Diseña el arreglo

Índice 0	Índice 1	Índice 2	Índice 3	Índice 4	Índice 5
3.5	4.2	2.8	5.0	3.9	4.5

Tercera nota ingresada: El valor es 2.8 y está ubicada en el índice 2

Pseudocódigo para el último estudiante: `notas[longitud(notas) - 1]`

2.2 Direcciones de memoria

`notas[0] = 0x2000 + (0 * 4) = 0x2000`

`notas[3] = 0x2000 + (3 * 4) = 0x2000 + 12 bytes -> 0x200C (El 12 es C en hexadecimal)`

`notas[5] = 0x2000 + (5 * 4) = 0x2000 + 20 bytes -> 0x2014 (El 20 es 14 en hexadecimal)`

Explicación: Es el $O(1)$ el sistema no busca la nota. Al pedir `notas[3]`, el procesador multiplica instantáneamente $3 * 4$, le suma el resultado a la base y sabe de inmediato que debe ir a la dirección física 0x200C. Al ser una simple operación aritmética, toma el mismo microsegundo leer la primera casilla que la última.

2.3 Búsqueda lineal vs. binaria

Simulación Lineal: Revisa el 15 (no), el 18 (no), el 20 (no), el 23 (no), el 27 (no) y finalmente el 31 (encontrado). Hizo 6 comparaciones.

Simulación Binaria: Parte el arreglo a la mitad (índice 3, valor 23). Como $31 > 23$, descarta toda la mitad izquierda. Con lo que queda a la derecha, calcula la nueva mitad (índice 5, valor 31). Compara y lo encuentra. Hizo solo 2 comparaciones.

¿Por qué requiere orden? Porque la búsqueda binaria se basa en la lógica de descartar lados. Si le dices que el número del medio es un 20, asume con ojos cerrados que los mayores están a la derecha. Si estuviera desordenado, terminaría descartando números válidos por error.

Para 1 millón de datos: Seguiría usando binaria. La lineal podría llegar a tardar un millón de pasos si el dato no está, mientras que la binaria resolvería la búsqueda en un máximo de 20 pasos gracias al descarte exponencial ($O(\log n)$).

2.4 Insertar un elemento

Al final: Si hay espacio libre, cuesta nada ($O(1)$), solo pones el dato en el casillero vacío. Pero si el arreglo está lleno, toca crear un arreglo gigante nuevo en otra parte de la memoria y copiar todos los datos viejos uno por uno, lo que sube el costo a $O(n)$.

En el medio (Insertar 21): Cuesta $O(n)$ porque no puedes simplemente sobrescribir el 23. Tienes que empujar el 23 y el 27 una casilla hacia la derecha para hacer espacio en blanco y luego colocar el 21 en su lugar. Mover datos es costoso.

Parte 3 — Arreglos 2D

3.1 Diseña la matriz

Salón	Col 0	Col 1	Col 2	Col 3
Fila 0	[0][0]	[0][1]	[0][2]	[0][3]
Fila 1	[1][0]	[1][1]	[1][2]	[1][3]
Fila 2	[2][0]	[2][1]	[2][2]	[2][3]

Pseudocódigo para fila 2, columna 3: `salon[2][3]`

Total de casillas: Hay 12 casillas. La regla general es multiplicar la cantidad de filas por la cantidad de columnas.

3.2 De 2D a memoria (row-major)

Aplanamiento en la RAM: Físicamente, la memoria guarda la matriz como una sola tira larga: `salon[0][0]`, `salon[0][1]`, `salon[0][2]`, `salon[0][3]`, `salon[1][0]`, ...

- $\text{salon}[1][2] = 0x1000 + ((1 * 4 + 2) * 4) = 0x1000 + 24 \rightarrow 0x1018$
- $\text{salon}[0][0] = 0x1000 + ((0 * 4 + 0) * 4) = 0x1000 + 0 \rightarrow 0x1000$
- $\text{salon}[2][3] = 0x1000 + ((2 * 4 + 3) * 4) = 0x1000 + 44 \rightarrow 0x102C$

3.3 Recorrido con ciclos anidados

contador = 0

Para f desde 0 hasta 2 Hacer:

Para c desde 0 hasta 3 Hacer:

Si salon[f][c] == 1 Entonces:

contador = contador + 1

Fin Si

Fin Para

Fin Para

Imprimir contador

Complejidad Big O: $O(f * c)$. Es así porque el ciclo de afuera obliga al ciclo de adentro a dar una vuelta completa por cada fila, pasando obligatoriamente por todas y cada una de las casillas de la estructura.

Si se duplican: El tiempo de ejecución se multiplica por 4. Al duplicar tanto el ancho como el alto de una cuadrícula, el área total de búsqueda se cuadruplica.

3.4 Conectando todo

Estos tres temas son el pilar para escribir código que no bloquee las computadoras. Los arreglos 1D y 2D nos dan una forma ordenada y limpia de estructurar la información en la memoria del equipo de forma contigua, lo que garantiza lecturas instantáneas de complejidad $O(1)$. Sin embargo, cuando dejamos de leer casillas fijas y empezamos a operar con esos datos, entra en juego Big O. Entender la complejidad es lo que nos permite elegir con criterio: nos dice cuándo vale la pena pagar el costo de ordenar un arreglo para poder aplicar búsquedas binarias ultrarrápidas ($O(\log n)$), o cuándo una matriz justifica sus ciclos anidados de $O(n^2)$ a cambio de representar mejor un problema de la vida real.